ESGIS BÉNIN

Filière Systèmes, Réseaux et Cloud (SRC)

TRAVAUX PRATIQUES

TP 2

Installation et configuration d’un serveur DNS avec BIND9

Groupe n°1
HOUAGBEME Amos

Année académique : 2026-2027
Date de remise : 17/10/2026

Sommaire

1. Introduction et objectifs..... 2
2. Environnement et architecture du laboratoire..... 2
3. Installation de BIND9..... 4
4. Configuration de la zone directe..... 4
5. Configuration de la zone inverse..... 6
6. Vérification des fichiers de configuration..... 7
7. Tests DNS..... 8
8. Configuration et tests du client..... 12
9. Conclusion..... 15

1. Introduction et objectifs

1.1 Le rôle du DNS

Le DNS (Domain Name System) relie un nom, par exemple `www.linux.local`, à une adresse IP. On n'a donc pas besoin de retenir des suites de chiffres, et une machine peut changer d'adresse sans que son nom bouge. D'autres services s'appuient dessus : pour la messagerie, c'est un enregistrement MX qui dit quel serveur reçoit le courrier d'un domaine.

Les noms sont rangés par zones. Le serveur qui gère une zone y fait autorité. On parle de résolution directe quand on part du nom pour trouver l'adresse, et de résolution inverse quand on fait le chemin contraire.

1.2 Le rôle de BIND9

BIND9 est un serveur DNS très répandu. Il peut faire autorité sur des zones locales et, en même temps, répondre aux clients comme résolveur. Son service s'appelle `named`. Sous Ubuntu, il s'installe avec le paquet `bind9` et sa configuration se trouve dans `/etc/bind/`.

1.3 Objectifs du TP

Nous devions installer BIND9 et vérifier qu'il tourne, configurer la zone directe `linux.local` ainsi que la zone inverse du réseau `192.168.1.0/24`, y déclarer les enregistrements demandés, contrôler les fichiers, puis tester la résolution depuis le serveur et depuis un client.

2. Environnement et architecture du laboratoire

2.1 Environnement de virtualisation

Le laboratoire repose sur deux machines virtuelles Ubuntu placées sur le même réseau virtuel, `192.168.1.0/24`. Ce réseau est créé avec KVM/libvirt et géré depuis virt-manager.

2.2 Les machines

DNS-SRV est le serveur DNS :

- adresse `192.168.1.2/24` sur l'interface `enp1s0` ;
- BIND9 installé dessus ;
- réseau géré par Netplan, dans `/etc/netplan/01-dns-server.yaml` ;
- il utilise `127.0.0.1` comme serveur DNS, c'est-à-dire lui-même.

CLIENT-UBUNTU sert de client de test :

- adresse `192.168.1.3/24` sur `enp1s0` ;
- outils de test `dig` et `resolvectl` ;
- réseau géré par NetworkManager (fichier Netplan `/etc/netplan/01-network-manager-all.yaml`) ;
- il doit utiliser `192.168.1.2` comme serveur DNS.

Sur les captures d'écran, les deux machines apparaissent sous les noms `server-dns` et `PC-client`.

2.3 Configuration réseau du serveur

La configuration réseau de DNS-SRV est dans `/etc/netplan/01-dns-server.yaml`. C'est le fichier que nous avons utilisé pendant le TP :

```
network:
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false
      addresses:
        - 192.168.1.2/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 127.0.0.1
```

L'adresse est statique (`dhcp4: false`), ce qui convient à un serveur dont l'adresse ne doit pas changer. Le serveur se prend lui-même comme serveur DNS (`127.0.0.1`), ce qui permet de tester les zones avec `dig @127.0.0.1`. La route par défaut passe par `192.168.1.1`. Cette passerelle doit exister sur le réseau virtuel pour sortir du réseau, mais les deux machines sont dans le même sous-réseau et leur communication n'en dépend pas.

La figure 1 montre le fichier tel qu'il s'affiche sur le serveur (avec `sudo`, car il n'est pas lisible sans droits d'administrateur) et l'adresse portée par `enp1s0`.

```
stone@server-dns:~$ cat /etc/netplan/01-dns-server.yaml
cat: /etc/netplan/01-dns-server.yaml: Permission denied
stone@server-dns:~$ sudo cat /etc/netplan/01-dns-server.yaml
network:
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false
      addresses:
        - 192.168.1.2/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 127.0.0.1
stone@server-dns:~$ ip -4 addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.1.2/24 brd 192.168.1.255 scope global enp1s0
       valid_lft forever preferred_lft forever
stone@server-dns:~$ _
```

Figure 1 : fichier Netplan de DNS-SRV et adresse de l'interface `enp1s0`.

2.4 Configuration réseau du client

CLIENT-UBUNTU a l'adresse `192.168.1.3/24` sur `enp1s0`. Il est géré par NetworkManager (fichier Netplan `/etc/netplan/01-network-manager-all.yaml`) et doit utiliser `192.168.1.2` comme serveur DNS. Sa configuration DNS, et notamment la modification de `/etc/resolv.conf`, est détaillée à la section 8.

3. Installation de BIND9

3.1 Commandes utilisées

```
sudo apt update
sudo apt install bind9 bind9-utils dnsutils -y
systemctl status bind9
```

3.2 Ce que fait chaque élément

- `apt update` met à jour la liste des paquets disponibles ;
- `bind9` est le serveur DNS lui-même (service `named`), avec ses fichiers dans `/etc/bind/` ;
- `bind9-utils` apporte `named-checkconf` et `named-checkzone`, utilisés à la section 6 ;
- `dnsutils` fournit `dig`, pour interroger le DNS ;
- `systemctl status bind9` affiche l'état du service et les derniers messages de son journal.

3.3 État du service

La figure 2 montre que `named` est `active (running)` et activé au démarrage. Le journal contient des messages « network unreachable resolving » concernant des adresses IPv6 de serveurs racine. Ces adresses ne sont pas joignables depuis notre laboratoire, ce qui touche seulement la résolution vers l'extérieur et pas les zones du TP.

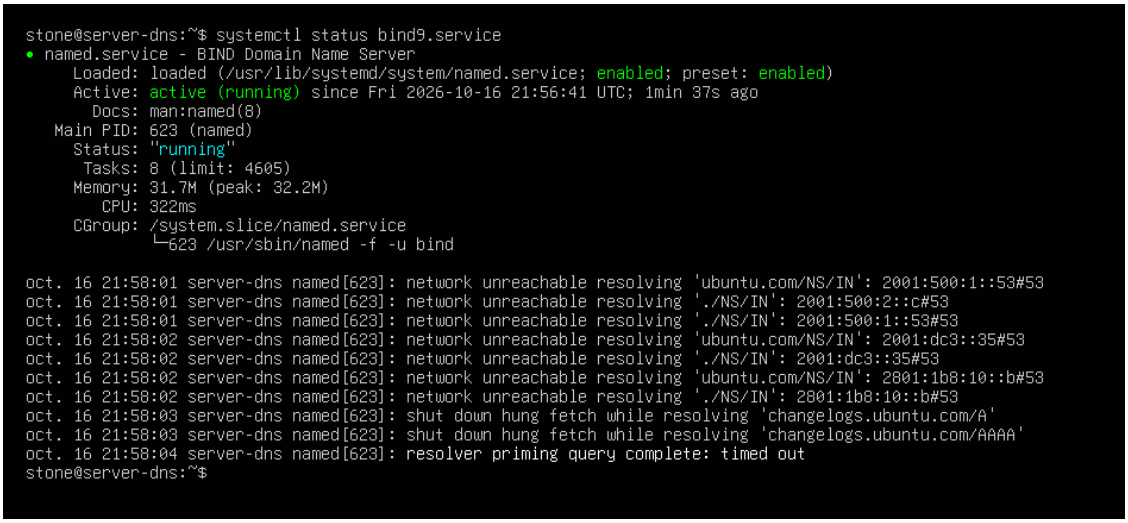


Figure 2 : état du service BIND9 sur DNS-SRV (`systemctl status bind9.service`).

4. Configuration de la zone directe

4.1 Déclaration de la zone

La zone est déclarée dans `/etc/bind/named.conf.local`. Le serveur en est le maître (`type master`) et le fichier de zone s'appelle `db.linux.local` :

```
zone "linux.local" {
    type master;
    file "/etc/bind/db.linux.local";
};
```

4.2 Fichier de zone

Contenu du fichier `/etc/bind/db.linux.local` :

```
$TTL 86400
@ IN SOA dns1.linux.local. admin.linux.local. (
    2026101401
    3600
    1800
    604800
    86400
)

@     IN NS     dns1.linux.local.
dns1  IN A      192.168.1.2
www   IN A      192.168.1.10
www   IN AAAA   2001:db8::10
web   IN CNAME  www.linux.local.
@     IN MX 10  mail.linux.local.
mail  IN A      192.168.1.20
```

La figure 3 montre que les fichiers du serveur correspondent bien à ce contenu.

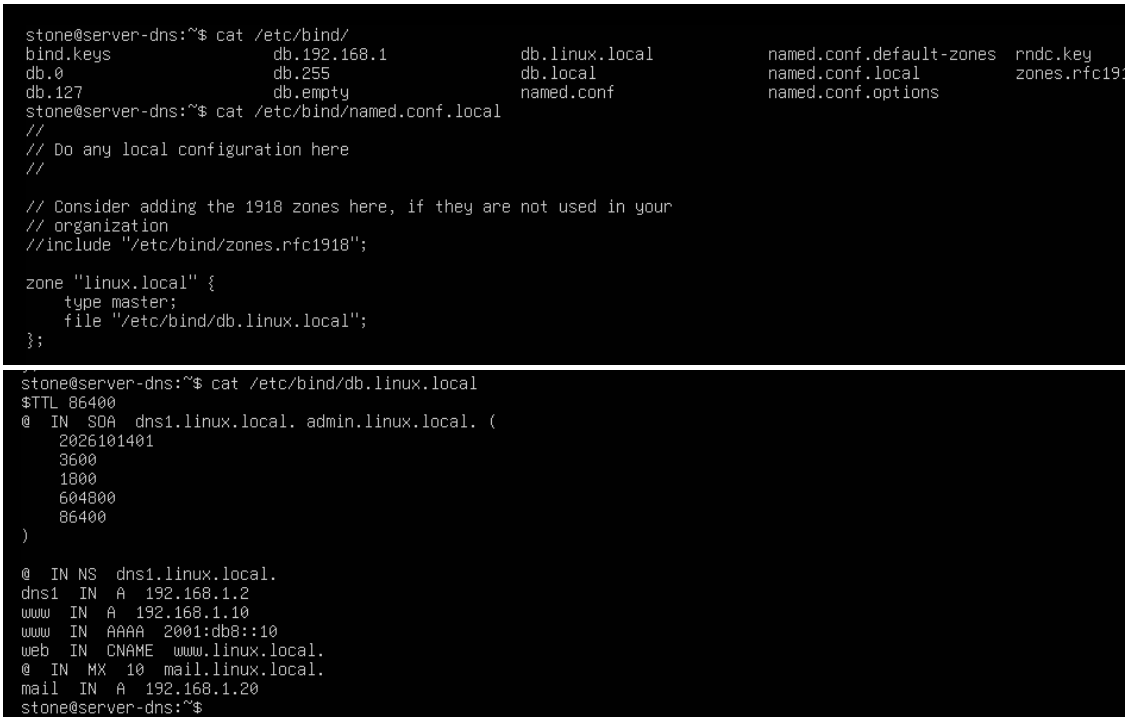


Figure 3 : zone directe sur DNS-SRV, `named.conf.local` *puis* `db.linux.local`.

4.3 Les enregistrements

- SOA : décrit la zone (serveur principal, adresse de contact de l'administrateur, numéro de série et durées de gestion) ;
- NS : désigne le serveur de noms de la zone, ici `dns1.linux.local.` ;
- A : associe un nom à une adresse IPv4. Nous en avons trois, `dns1`, `www` et `mail` ;
- AAAA : même chose pour une adresse IPv6, ici pour `www` ;
- CNAME : un alias. `web.linux.local` est un autre nom pour `www.linux.local` ;

- MX : indique le serveur de messagerie du domaine. Le 10 est la priorité (la plus petite valeur passe en premier), et la cible doit être un nom qui a lui-même un enregistrement A, ici `mail`.

Les valeurs du SOA de notre fichier :

- numéro de série `2026101401` : la version de la zone, au format date + numéro (AAAAMMJJNN) ;
- refresh `3600` : le délai, en secondes, entre deux vérifications d'un serveur secondaire ;
- retry `1800` : le délai avant une nouvelle tentative en cas d'échec ;
- expire `604800` : la durée au bout de laquelle un secondaire cesse de faire autorité s'il ne joint plus le primaire ;
- TTL négatif `86400` : la durée de mise en cache des réponses négatives.

Le `$TTL 86400` en tête de fichier est la durée de vie par défaut des enregistrements, soit 24 heures.

4.4 Précisions

- `192.168.1.10` est déclarée dans le DNS, mais notre laboratoire ne contient aucun serveur web. Le DNS donne l'adresse sans vérifier qu'une machine y répond.
- `192.168.1.20` n'existe que pour servir de cible à l'enregistrement MX. Rien ne prouve qu'un serveur de messagerie s'y trouve.
- `2001:db8::10` appartient à un préfixe réservé à la documentation. L'enregistrement AAAA ne prouve donc pas que l'IPv6 fonctionne.
- Le numéro de série doit être augmenté à chaque modification du fichier de zone, sinon les serveurs secondaires ne voient pas le changement.

5. Configuration de la zone inverse

5.1 Déclaration de la zone

La zone inverse est déclarée dans le même fichier, `/etc/bind/named.conf.local`, à la suite de la zone directe :

```
zone "1.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.192.168.1";
};
```

5.2 Nom de la zone inverse

Les zones inverses se trouvent sous `in-addr.arpa`. Pour fabriquer le nom, on prend la partie réseau de l'adresse et on inverse l'ordre des octets : `192.168.1` devient `1.168.192`, d'où `1.168.192.in-addr.arpa`. Dans cette zone, c'est le dernier octet qui sert de nom, donc `192.168.1.10` correspond à `10.1.168.192.in-addr.arpa`.

5.3 Fichier de zone inverse

Contenu du fichier `/etc/bind/db.192.168.1` :

```
$TTL 86400
@ IN SOA dns1.linux.local. admin.linux.local. (
    2026101401
    3600
    1800
    604800
    86400
)

@   IN NS   dns1.linux.local.
10  IN PTR  www.linux.local.
```

L'enregistrement PTR assure la résolution inverse : la ligne `10 IN PTR www.linux.local.` veut dire que `192.168.1.10` correspond à `www.linux.local`. Le nom est écrit en entier, avec le point final, sinon BIND9 y ajouterait le nom de la zone. Le résultat du test PTR est à la section 7.1 (figure 10).

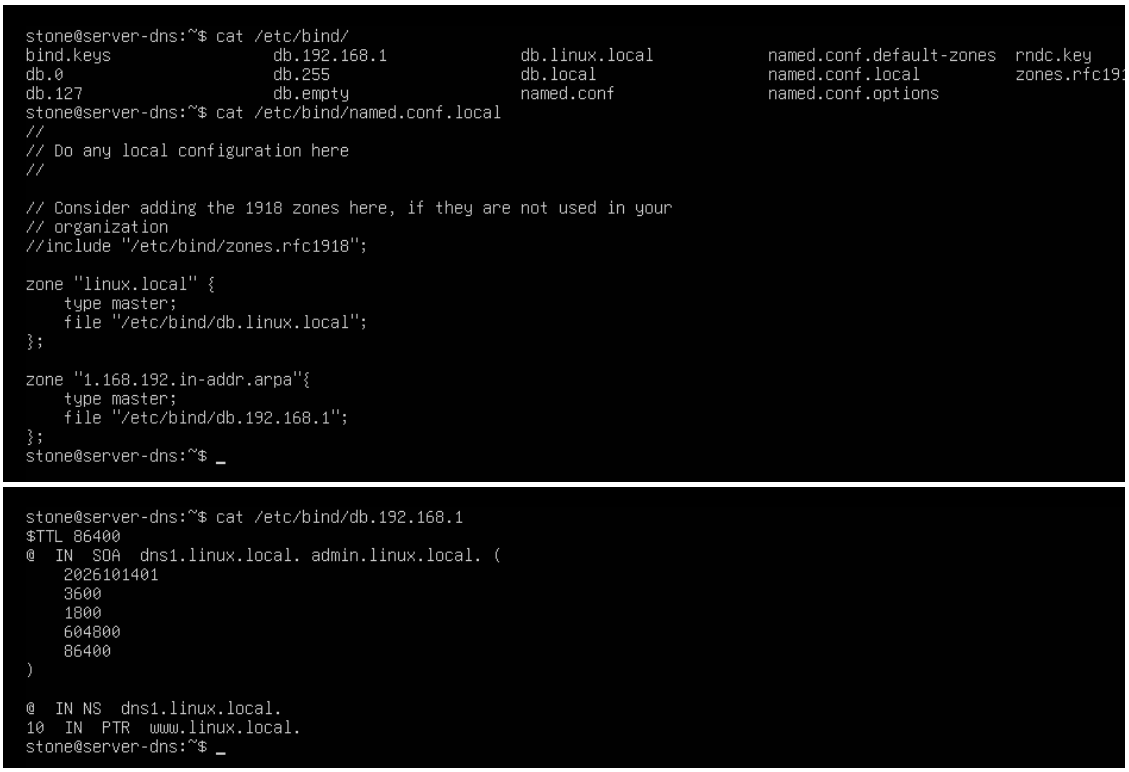


Figure 4 : zone inverse sur DNS-SRV, `named.conf.local` avec les deux zones, puis `db.192.168.1`.

6. Vérification des fichiers de configuration

Avant de recharger le service, nous avons vérifié la syntaxe de la configuration et des deux zones, pour ne pas perturber un service qui tourne à cause d'une faute dans un fichier.

```
sudo named-checkconf
sudo named-checkzone linux.local /etc/bind/db.linux.local
sudo named-checkzone 1.168.192.in-addr.arpa /etc/bind/db.192.168.1
sudo systemctl reload bind9
```

- `named-checkconf` contrôle la syntaxe des fichiers de configuration, dont `named.conf.local` ;
- `named-checkzone` contrôle un fichier de zone et affiche le numéro de série chargé ;
- `systemctl reload bind9` fait relire la configuration sans arrêter le service.

La figure 5 montre ce que nous avons obtenu :

- `named-checkconf` n'affiche rien, donc pas d'erreur de syntaxe ;
- pour la zone directe : `zone linux.local/IN: loaded serial 2026101401`, puis `OK` ;
- pour la zone inverse : `zone 1.168.192.in-addr.arpa/IN: loaded serial 2026101401`, puis `OK` ;
- le rechargement ne renvoie pas d'erreur, le statut reste `active (running)` et le journal indique `all zones loaded`.

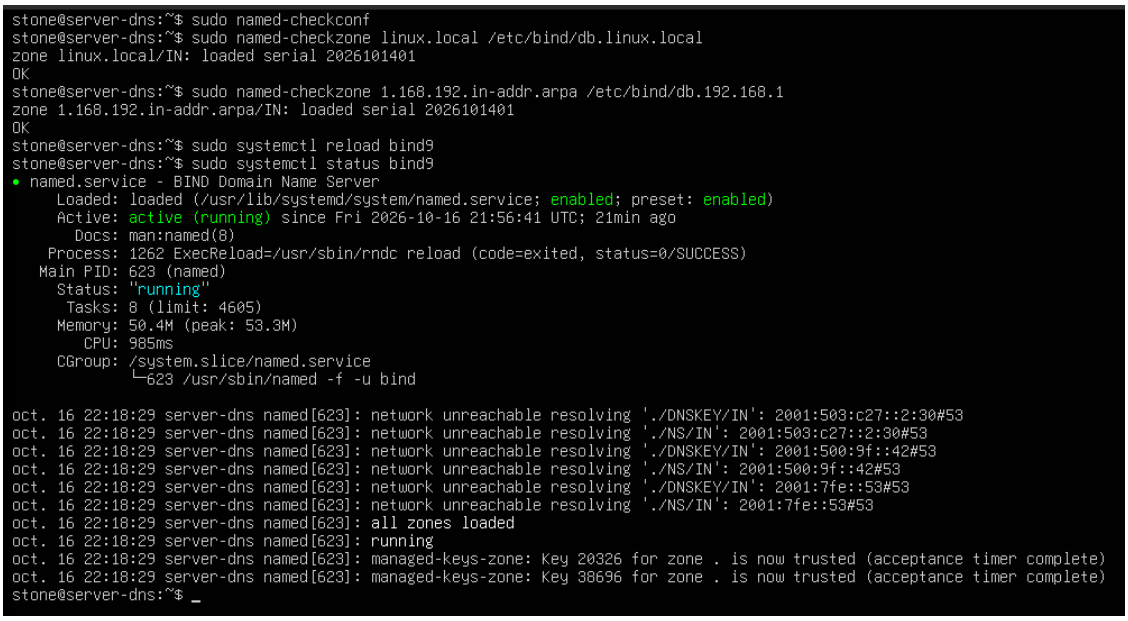


Figure 5 : vérification des fichiers, rechargement et état du service.

7. Tests DNS

7.1 Tests sur le serveur

Nous avons d'abord testé chaque type d'enregistrement depuis DNS-SRV, en interrogeant `127.0.0.1` :

```
dig @127.0.0.1 www.linux.local A
dig @127.0.0.1 www.linux.local AAAA
dig @127.0.0.1 web.linux.local CNAME
dig @127.0.0.1 linux.local MX
dig -x 192.168.1.10 @127.0.0.1
```

Dans chaque réponse, on regarde trois choses : le statut (`NOERROR` si la requête a abouti, `NXDOMAIN` si le nom n'existe pas), le drapeau `aa` (la réponse vient d'un serveur qui fait autorité sur la zone) et la section `ANSWER`. Voici ce que nous avons relevé :

- `www.linux.local` en A : `NOERROR`, `aa`, `www.linux.local. 86400 IN A 192.168.1.10` ;

- `www.linux.local` en AAAA : `NOERROR`, `aa`, `www.linux.local. 86400 IN AAAA 2001:db8::10` ;
- `web.linux.local` en CNAME : `NOERROR`, `aa`, `web.linux.local. 86400 IN CNAME www.linux.local.` ;
- `linux.local` en MX : `NOERROR`, `aa`, `linux.local. 86400 IN MX 10 mail.linux.local.`, et dans la section ADDITIONAL `mail.linux.local. 86400 IN A 192.168.1.20` ;
- `dig -x 192.168.1.10` : `NOERROR`, `aa`, `10.1.168.192.in-addr.arpa. 86400 IN PTR www.linux.local.`

Ces tests (figures 6 à 10) montrent que BIND9 sert bien la zone directe et la zone inverse. Ils ne prouvent pas qu'il existe des machines aux adresses `192.168.1.10` et `192.168.1.20`, et ils ont été faits sur le serveur, pas depuis le client. Sur chaque capture, on voit aussi l'avertissement de `dig` à propos du suffixe `.local`, expliqué à la section 8.5.

```
stone@server-dns:~$ dig @127.0.0.1 www.linux.local A

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @127.0.0.1 www.linux.local A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 25664
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: bd8e85d578bc3c2d010000006ad2a37d5927469b3025aa24 (good)
;; QUESTION SECTION:
;www.linux.local.               IN      A

;; ANSWER SECTION:
www.linux.local.        86400   IN      A       192.168.1.10

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Fri Oct 16 22:21:49 UTC 2026
;; MSG SIZE  rcvd: 88

stone@server-dns:~$ _
```

Figure 6 : `dig @127.0.0.1 www.linux.local A`.

```
stone@server-dns:~$ dig @127.0.0.1 www.linux.local AAAA

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @127.0.0.1 www.linux.local AAAA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 16536
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b9861a160da6748b010000006ad2a398614928b78afb457a (good)
;; QUESTION SECTION:
;www.linux.local.               IN      AAAA

;; ANSWER SECTION:
www.linux.local.        86400   IN      AAAA    2001:db8::10

;; Query time: 1 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Fri Oct 16 22:22:16 UTC 2026
;; MSG SIZE  rcvd: 100

stone@server-dns:~$
```

Figure 7 : `dig @127.0.0.1 www.linux.local AAAA`.

```
stone@server-dns:~$ dig @127.0.0.1 web.linux.local CNAME

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @127.0.0.1 web.linux.local CNAME
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 20750
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 39d719d765a67b93010000006ad2a3aa0fb81e81a2a2fd59 (good)
;; QUESTION SECTION:
;web.linux.local.               IN      CNAME

;; ANSWER SECTION:
web.linux.local.        86400   IN      CNAME   www.linux.local.

;; Query time: 1 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Fri Oct 16 22:22:34 UTC 2026
;; MSG SIZE  rcvd: 90

stone@server-dns:~$ _
```

Figure 8 : `dig @127.0.0.1 web.linux.local CNAME`.

```
stone@server-dns:~$ dig @127.0.0.1 linux.local MX

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @127.0.0.1 linux.local MX
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 1961
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 7fa2a978e8f4c53e010000006ad2a3b8db15b58d3ec11406 (good)
;; QUESTION SECTION:
;linux.local.                   IN      MX

;; ANSWER SECTION:
linux.local.            86400   IN      MX      10 mail.linux.local.

;; ADDITIONAL SECTION:
mail.linux.local.       86400   IN      A       192.168.1.20

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Fri Oct 16 22:22:48 UTC 2026
;; MSG SIZE  rcvd: 105

stone@server-dns:~$
```

Figure 9 : `dig @127.0.0.1 linux.local MX`.

```
stone@server-dns:~$ dig -x 192.168.1.10 @127.0.0.1

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> -x 192.168.1.10 @127.0.0.1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60983
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 35f3b223dbe05c24010000006ad2a3d0d30af4260030d2d3 (good)
;; QUESTION SECTION:
;10.1.168.192.in-addr.arpa.     IN      PTR

;; ANSWER SECTION:
10.1.168.192.in-addr.arpa. 86400 IN     PTR     www.linux.local.

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Fri Oct 16 22:23:12 UTC 2026
;; MSG SIZE  rcvd: 111

stone@server-dns:~$ _
```

Figure 10 : `dig -x 192.168.1.10 @127.0.0.1`.

7.2 Test depuis le client

Depuis CLIENT-UBUNTU, nous avons interrogé explicitement le serveur `192.168.1.2` :

```
dig @192.168.1.2 www.linux.local A
```

La réponse obtenue est :

```
www.linux.local. 86400 IN A 192.168.1.10
```

`86400` est le TTL en secondes (24 h), `IN` la classe Internet, `A` le type d'enregistrement et `192.168.1.10` l'adresse renvoyée, la même que dans la zone. Le serveur a donc bien fourni l'adresse déclarée pour `www.linux.local`. La figure 11 montre aussi `status: NOERROR`, le drapeau aa et `SERVER: 192.168.1.2#53`, ce qui confirme que la réponse vient de DNS-SRV.

```
cassian@PC-client:~$ dig @192.168.1.2 www.linux.local

; <<>> DiG 9.18.30-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.1.2 www.linux.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54347
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: f55984f87e8d24d6010000006ad2a4939716b0c9cf986433 (good)
;; QUESTION SECTION:
;www.linux.local.               IN      A

;; ANSWER SECTION:
www.linux.local.        86400   IN      A       192.168.1.10

;; Query time: 0 msec
;; SERVER: 192.168.1.2#53(192.168.1.2) (UDP)
;; WHEN: Fri Oct 16 23:26:27 WAT 2026
;; MSG SIZE  rcvd: 88

cassian@PC-client:~$
```

Figure 11 : `dig @192.168.1.2 www.linux.local A` depuis CLIENT-UBUNTU.

8. Configuration et tests du client

8.1 Le client

Le client est CLIENT-UBUNTU (`192.168.1.3/24`, interface `enp1s0`), géré par NetworkManager. Il sert à tester la résolution DNS fournie par DNS-SRV.

8.2 État du client avant modification

Nous avons relevé la configuration du client avec ces commandes :

```
ip -4 addr show
ip route
resolvectl status
cat /etc/resolv.conf
```

La figure 12 donne le résultat :

- adresse du client : `192.168.1.3/24` sur `enp1s0` ;
- routes : route par défaut via `192.168.1.1` et réseau local `192.168.1.0/24` ;
- serveur DNS de l'interface (`resolvectl status`) : `192.168.1.2` ;
- `/etc/resolv.conf` en mode stub, géré par `systemd-resolved`, avec `nameserver 127.0.0.53`, `options edns0 trust-ad` et `search ..`

Le serveur DNS configuré sur l'interface est donc bien `192.168.1.2`, mais `/etc/resolv.conf`, que lisent les applications, pointe vers le relais local `127.0.0.53` de `systemd-resolved`.

```
cassian@PC-client:~$ ip -4 addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.1.3/24 brd 192.168.1.255 scope global noprefixroute enp1s0
       valid_lft forever preferred_lft forever
cassian@PC-client:~$ ip route
default via 192.168.1.1 dev enp1s0 proto static metric 100
192.168.1.0/24 dev enp1s0 proto kernel scope link src 192.168.1.3 metric 100
cassian@PC-client:~$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (enp1s0)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.1.2
       DNS Servers: 192.168.1.2
cassian@PC-client:~$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
cassian@PC-client:~$
```

Figure 12 : configuration du client avant modification de `/etc/resolv.conf`.

8.3 Requête explicite et requête par défaut

`dig @192.168.1.2 www.linux.local A` interroge directement le serveur indiqué : on teste le serveur. `dig www.linux.local A`, sans `@`, utilise le résolveur configuré sur le client (`/etc/resolv.conf`) : on teste la configuration du client. Si la première réussit et pas la seconde, c'est que le serveur fonctionne mais que le client n'est pas orienté vers lui.

8.4 Modification de /etc/resolv.conf

Avant la modification, `dig www.linux.local A` interroge `127.0.0.53` et se termine par `status: SERVFAIL`, sans réponse (figure 13). Pour obtenir une résolution sans préciser le serveur dans la commande, nous avons modifié `/etc/resolv.conf` : la ligne `nameserver 127.0.0.53` a été mise en commentaire et la ligne `nameserver 192.168.1.2` ajoutée. La même commande interroge alors `192.168.1.2` et renvoie `192.168.1.10` avec `status: NOERROR` (figure 14).

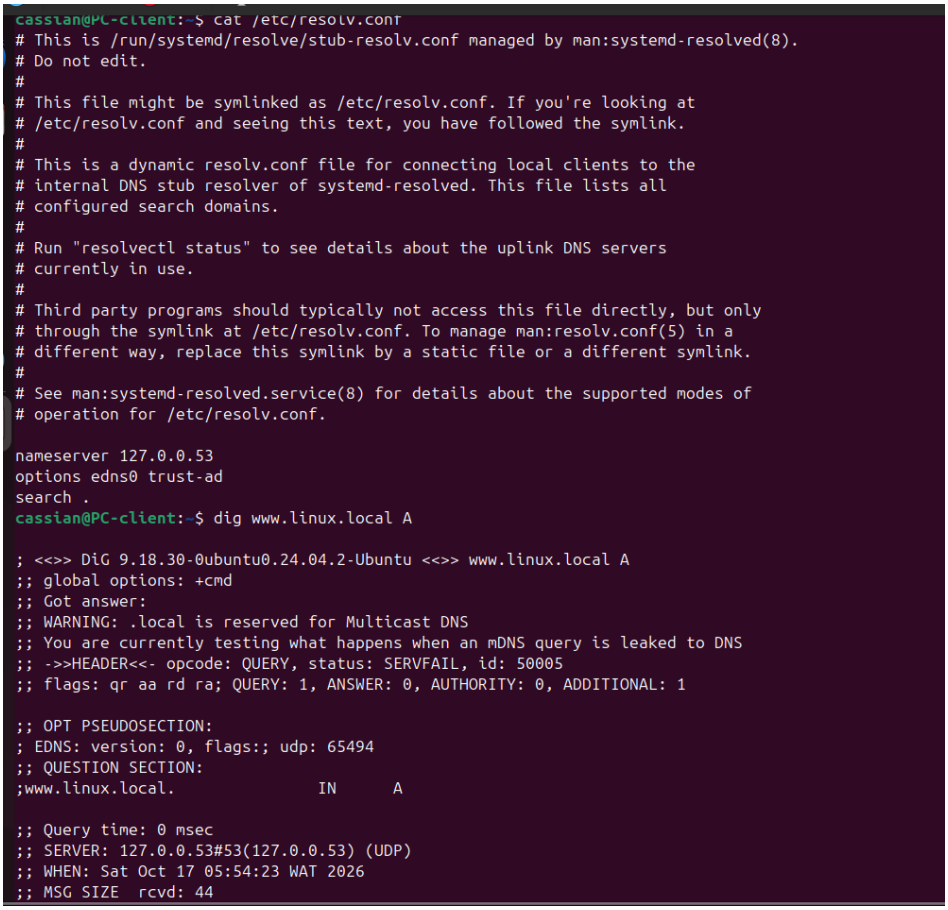


Figure 13 : avant modification, `/etc/resolv.conf` puis `dig www.linux.local A` (SERVFAIL).

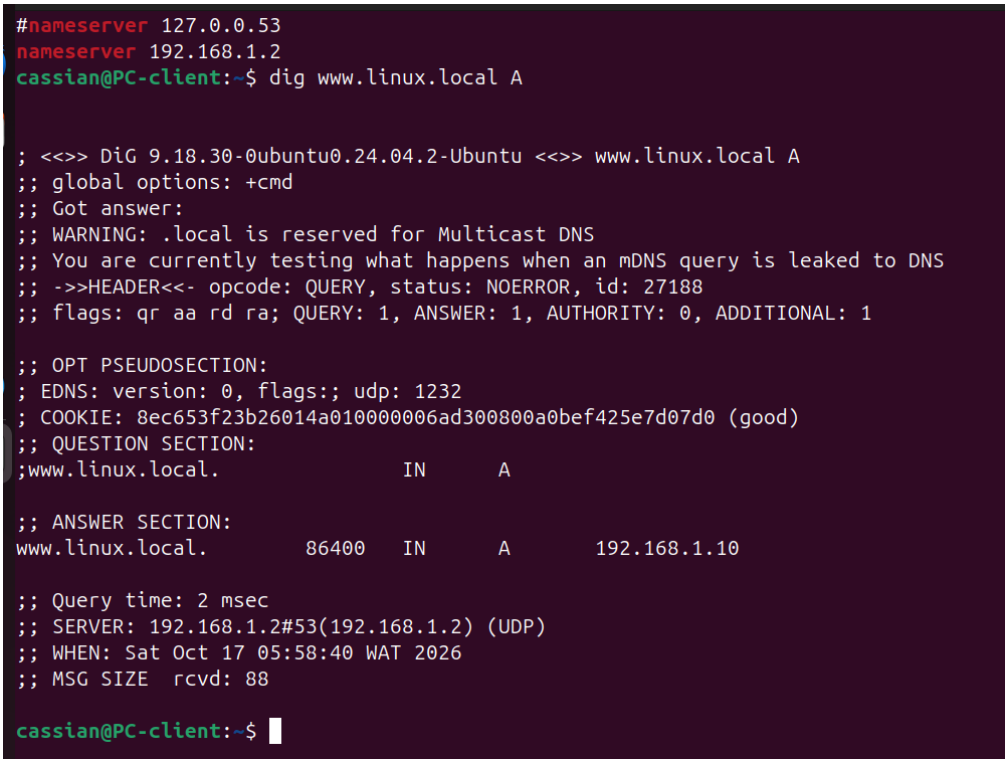


Figure 14 : après modification, `nameserver 192.168.1.2` puis `dig www.linux.local A` (192.168.1.10).

En résumé, pour la même commande `dig www.linux.local A` :

- avant, le serveur interrogé était `127.0.0.53` et le résultat `SERVFAIL` ;
- après, le serveur interrogé était `192.168.1.2` et le résultat `NOERROR` avec `192.168.1.10`.

Nous n'avons pas cherché la cause exacte du `SERVFAIL` initial. Par ailleurs, `/etc/resolv.conf` est géré automatiquement par le système, et notre modification peut être écrasée (redémarrage, reconnexion au réseau) : nous avons d'ailleurs dû la refaire. Une solution plus durable serait d'indiquer le serveur DNS `192.168.1.2` et le domaine `~linux.local` à NetworkManager avec `nmcli`, mais nous ne l'avons pas testée.

8.5 Le suffixe .local

Le domaine imposé par le TP est `linux.local`. Or le suffixe `.local` est réservé au mDNS (multicast DNS), utilisé par exemple avec Avahi ou `systemd-resolved`. Certains outils peuvent donc essayer de résoudre un nom en `.local` par mDNS au lieu d'interroger le serveur DNS. C'est pour cela que `dig` affiche l'avertissement « .local is reserved for Multicast DNS », visible sur nos captures. Le serveur répond quand même, mais d'autres commandes comme `ping`, ou un navigateur, pourraient se comporter autrement. Nous avons gardé `linux.local`, le domaine du TP. Dans `resolvectl status` (figure 12), le mDNS apparaît désactivé (`-mDNS`).

9. Conclusion

Dans ce TP, nous avons installé BIND9 sur une machine virtuelle Ubuntu (DNS-SRV, `192.168.1.2/24`), créé la zone directe `linux.local` et la zone inverse `1.168.192.in-addr.arpa`, déclaré des enregistrements SOA, NS, A, AAAA, CNAME, MX et PTR, puis vérifié les fichiers avec `named-checkconf` et `named-checkzone`. Les tests ont été faits sur le serveur et depuis un client Ubuntu (`192.168.1.3/24`).

Depuis le client, `dig @192.168.1.2 www.linux.local A` a renvoyé `192.168.1.10`. Sur le serveur, les tests A, AAAA, CNAME, MX et PTR ont tous donné le résultat attendu. Côté client, la résolution par défaut ne marchait pas tant que `/etc/resolv.conf` pointait vers `127.0.0.53` ; elle a fonctionné après l'ajout de `nameserver 192.168.1.2`, mais cette modification ne persiste pas.

Ce TP nous a permis de comprendre comment est construit un fichier de zone, à quoi sert chaque type d'enregistrement, pourquoi le numéro de série compte, comment on monte une zone inverse, et quelle différence il y a entre interroger un serveur précis et passer par le résolveur du client. Notre laboratoire ne contient ni serveur web ni serveur de messagerie : les adresses `192.168.1.10` et `192.168.1.20` sont seulement déclarées dans le DNS, et l'enregistrement AAAA ne prouve pas que l'IPv6 fonctionne.